Computational modeling

ESILV

2026

Ramine Nikoukhah

Course information

Course outline:

1. Introduction to Dynamical system modeling and script-based simulation
2. Block-diagram model construction and simulation
3. Event driven and hybrid systems
4. Physical system modeling: mechanical systems
5. Physical system modeling: Modelica
6. Simulation-based system optimization and AI

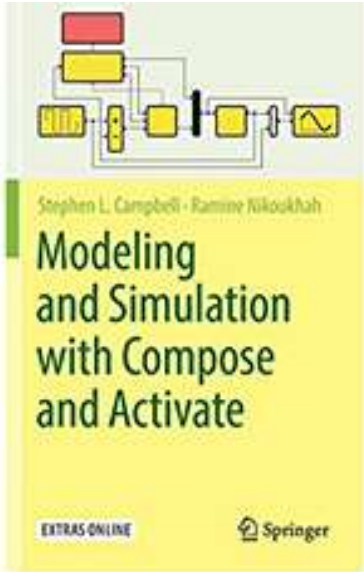
Course information

Modeling and Simulation Software TWIN ACTIVATE will be used for exercises: it can be downloaded from:

- https://web.altair.com/altair-student-edition#license-activation

Course grade is based on a final quiz

Some examples and exercises are from

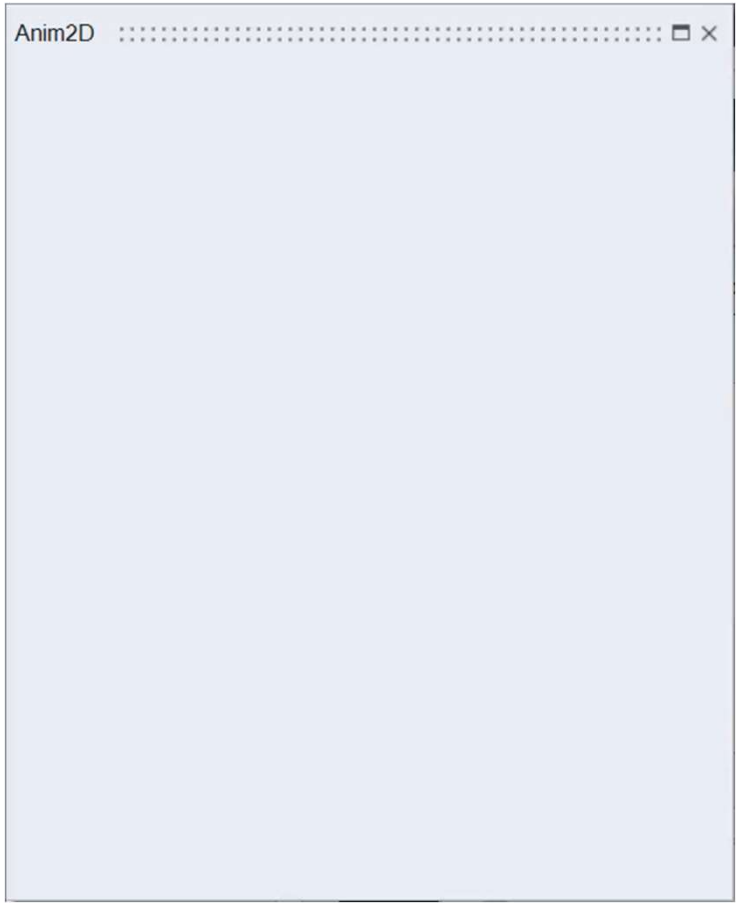


1 - Introduction to Dynamical system modeling and script-based simulation

Modeling and Simulation: what for?

- Study system behavior:
  - Explore system response
  - Evaluate stability and performance,
  - Measure robustness
  - Forecast, ...

- Design
  - Try different system configurations
  - Optimize model parameters
  - Controller design
  - Reduce cost and speed up the design and manufacturing process



What application domains?

- Industry: automotive, aeronautics, power plants, space, Telecom, transportation, manufacturing, robotics, chemical, pharmaceutical…
- Economy and Financial systems: macro and microeconomics, insurance, actuarial applications, taxation, stock market…
- Environment: climate, population dynamics, ecology…
- Scientific research: physics, biology, medicine…
- Others: video games, animation,…

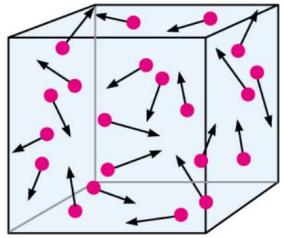
Dynamical system, what is it?

- System that evolves over time: its state is a function of time
- Examples
  - Predator-prey system: fox and rabbit population
  - Rocket, airplane, satellite…
  - Robot: movements to accomplish a task
  - Car engine: state reacts to user input and mechanical constraints
  - Epidemic: number of people infected by an infectious disease
  - Actuarial practice: solvency for a life insurance company
  - Electronic device: GPS system
  - Universe: expansion since big bang

May be natural, manufactured or virtual.

What is the state of a dynamical system?

- A complete representation of the system at a given time
- Given the current state, the knowledge of previous states provides no additional information regarding the future evolution of the system
  - System: gas particles in a box
  - State: position of the particles NO!
  - State: position and velocity



- System states include in general: position, speed, temperature, pressure, age, quantities, flow, configuration, voltage,…

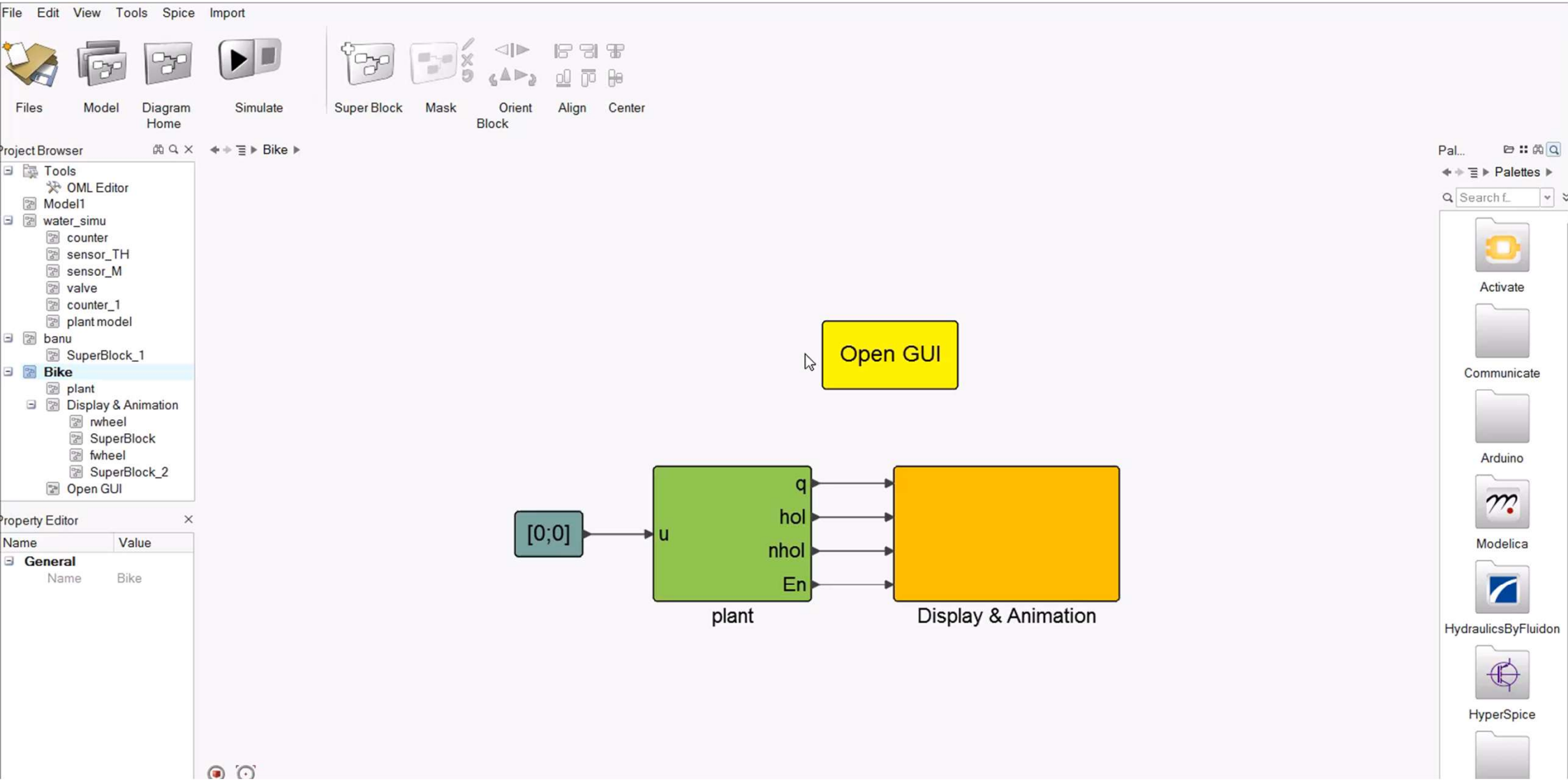
Model of a dynamical system

- A mathematical representation of the system exhibiting approximately the behavior of the true system
- Model equations obtained often based on physical laws (mechanical, electrical, hydraulic, pneumatic,... systems)
- Models also from other theories and principles: economics, population dynamics, epidemiology...
- Models may also be constructed/identified based on observations

Often models obtained from a mixture of laws and identification

Mathematical models:

- Represent the dynamics of the system by a set of mathematical equations and relations
- Model complexity depends on system nature and complexity, and required precision
- Depending on the objective of modeling, different aspects of the system behavior must be captured
- Ideally the parameters of the mathematical model correspond to system physical identifiable parameters
- Example: study bike stability for different geometries


File
Edit
View
Tools
Spice
Import
Files
Model
Diagram Home
Simulate
Super Block
Mask
Orient Block
Align
Center
Project Browser
Bike
Tools
OML Editor
Model1
water_simu
counter
sensor_TH
sensor_M
valve
counter_1
plant model
banu
SuperBlock_1
Bike
plant
Display & Animation
rwheel
SuperBlock
fwheel
SuperBlock_2
Open GUI
Property Editor
Name
Value
General
Name
Bike
Open GUI
[0;0]
u
q
hol
nhol
En
plant
Display & Animation
Palettes
Activate
Communicate
Arduino
Modelica
HydraulicsByFluidon
HyperSpice

Mathematical models: dynamics may be modeled in continuous or discrete time

- Object falling subject to gravity: use of differential equation

$$\ddot{y} = -g, \;\; g = 9.81 m/s^2$$

- Inventory (stock): daily management

$$S_{k+1} = S_k + I_k - O_k$$

- Many systems include both continuous-time and discrete-time dynamics
  - Gearbox dynamics in continuous-time with “jumps” when gear shifting at discrete-times
  - Systems including continuous and discrete dynamics are called **Hybrid Systems**

Mathematical models: deterministic vs stochastic

- Some applications require modeling systems with probabilistic behavior: counter queue at the supermarket
- Additional modeling complexity: finding proper probability laws
- Stochastic models often used to estimate mean/average/representative quantities: for example, the average waiting time at the Supermarket queue to decide if a new counter is needed
- Used also to study robustness and fault tolerance

Mathematical models: continuous-time states

- State of a mathematical model may be finite or infinite dimensional depending on the nature of the model
- Ordinary differential equations, differential-algebraic equations: finite state

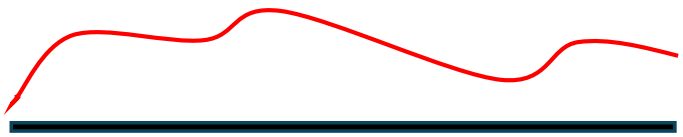
  Example of falling object:
  $\ddot{y} = -g$, if $y(t_0)$ and $\dot{y}(t_0)$ known, $y(t)$ fully specified for all t:

$$y(t) = y(t_0) + \dot{y}(t_0)(t - t_0) - \frac{1}{2}g(t - t_0)^2$$

- Partial differential equations, delay differential equations: infinite dimentsional state

  Example of the temperature distribution on a beam (heat equation)

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}$$



Mathematical models: Example of SIR

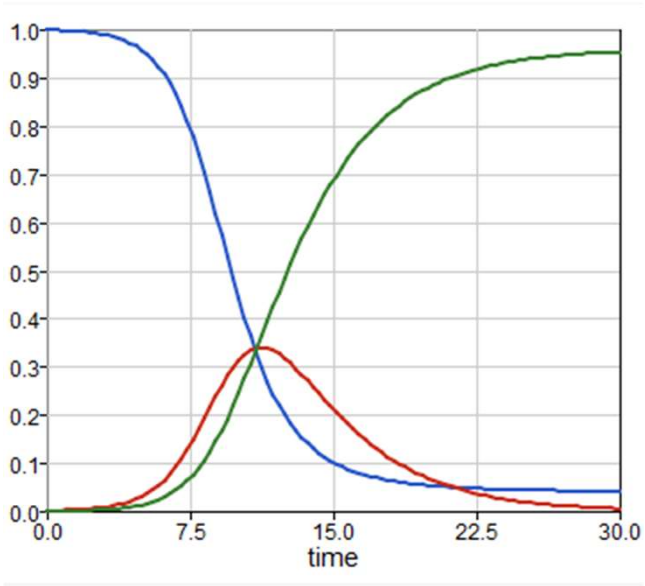
Spread of infectious disease

- *s(t)*: percentage of ***susceptible*** individuals
- *i(t)*: percentage of ***infected*** and contagious individuals
- *r(t)*: percentage of ***recovered*** individuals

- *a, b*: model parameters

Objective: study the evolution of the system as a function of parameter values

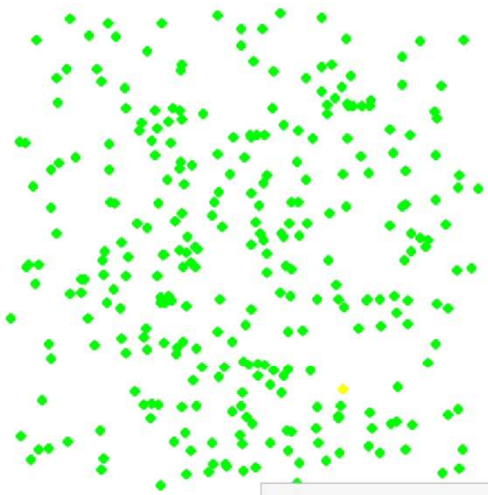
$$\dot{s} = -as(t)i(t),$$
$$\dot{i} = as(t)i(t) - bi(t),$$
$$\dot{r} = bi(t),$$



Stochastic model: statistical analysis

- Stochastic models simulated using random signals and computed averages
- Multiple simulations and/or simulation with large number of independent actors

- Example
  - Large number of individuals in a confined space, moving around
  - Infection can pass from one person to another if in close vicinity
  - Infected person can infect until it recovers after a delay

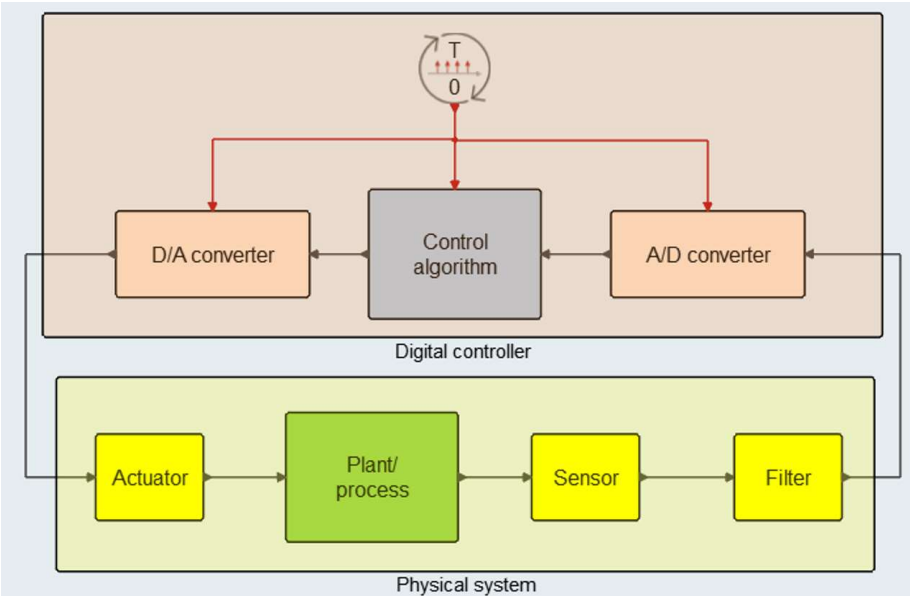


Mathematical models: time and state

- System states may have continuous and discrete values
- Time may also be continuous or discrete
- Examples:
  - Falling object (or any model represented by differential equations
    - State is continuous: the height and speed of the object
    - Time is continuous: state may have a different value at any instant of time
  - Supermarket queue
    - State is discrete: number of people waiting in the queue
    - Time is continuous: number can change at any instant in time
  - Digital controller
    - State is discrete: all digital values are discrete
    - Time is discrete: periodic clock

Mathematical models: mixed time and state

- Complex systems contain mix of time and state properties
- Important class of models: Continuous-time plant (physical system) controlled by digital controller
- Control algorithm implemented digitally running on a fixed frequency clock



Mathematical models considered here

- Models used for system analysis, optimization and controller design should be “simple”: finite state and possibly with small state size
- Robustness requirements impose the analysis for a family of systems, so the model used is a representative element of the family and not necessarily an exact model
- Such models need not be very precise but must capture relevant behaviors: nonlinearity, hybrid nature,...
- Only finite-state models, possibly for example through approximation of PDEs, considered here.

System of differential equations: how to determine state?

- System of differential equations have finite state, example

$$\ddot{y} = -g$$

- Initial conditions to specify uniquely the solution, $y(0) = y_0,\ \dot{y}(0) = v_0$
- Rewrite the system as a first order system (first derivatives only) by introducing additional variables (here the speed v):

$$\begin{aligned} \dot{y} &= v \\ \dot{v} &= -g \end{aligned}$$

- By applying systematically this technique (called state augmentation), any set of differential equation can be expressed as a first order system

System of differential equations: First order models

- First order system with states x_i:

$$\begin{aligned} \dot{x}_1 &= f_1(t, x_1, \cdots, x_n) \\ \dot{x}_2 &= f_2(t, x_1, \cdots, x_n) \\ &\vdots \\ \dot{x}_n &= f_n(t, x_1, \cdots, x_n) \end{aligned}$$

- Standard representation is:

$$\dot{x} = F(t, x)$$

with $x(t) = (x_1(t), x_2(t), \cdots, x_n(t))$: STATE VECTOR

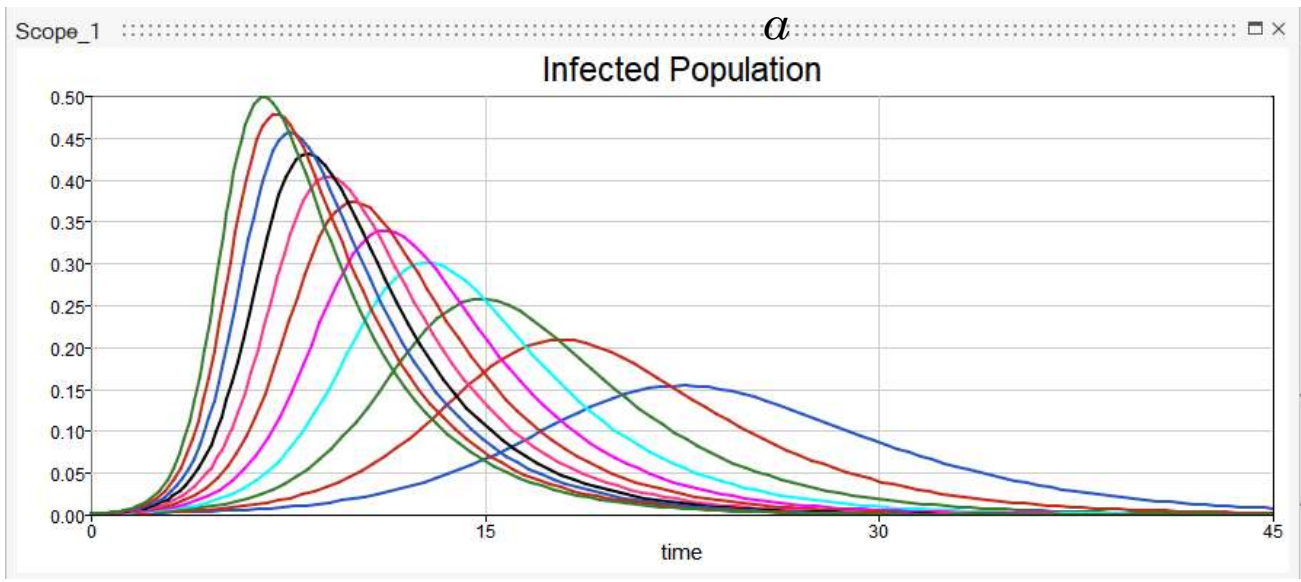
First order model: example

- Study effect of lockdown strategies in spread of virus
- SIR (initial conditions initial percentages *s(0), i(0), r(0)*)

$$\dot{s} = -as(t)i(t),$$
$$\dot{i} = as(t)i(t) - bi(t),$$
$$\dot{r} = bi(t),$$

- Parameter "a" depends on the lockdown strategy
- Find strategy to flatten the curve and avoid hospital saturation

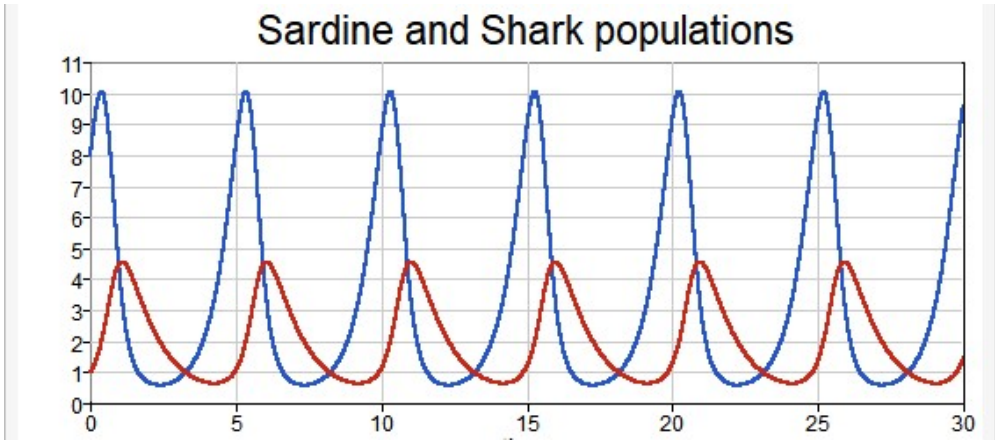
Varying parameter a



First order model: example with active control

- Model of Predator Prey system with fishing system.
  - x: sardine population
  - y: shark population

$$\begin{aligned} \dot{x} &= ax - bxy \\ \dot{y} &= cxy - dy \end{aligned}$$



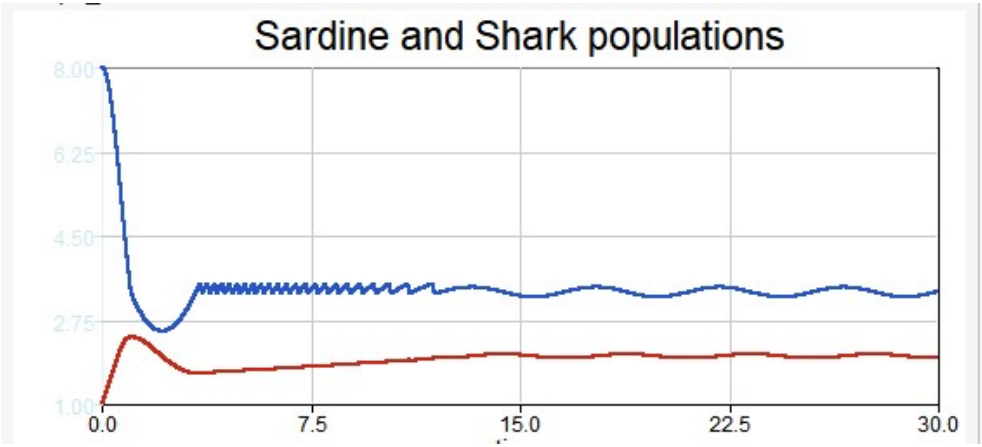
First order model: example with active control

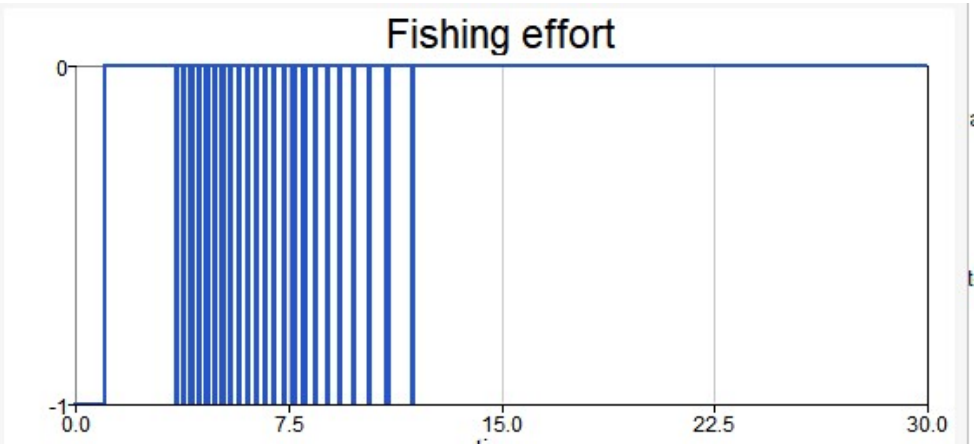
- Model of Predator Prey system with fishing (population control)

$$\begin{aligned} \dot{x} &= ax - bxy + F(t)x \\ \dot{y} &= cxy - dy \end{aligned}$$

- Harvesting (determine fishing season by finding optimal threshold)

$$F(t) = 0 \quad \text{if } x \text{ falls below } x_{\min} \text{ and } f \text{ is } x \text{ go beyond } x_{\max}$$





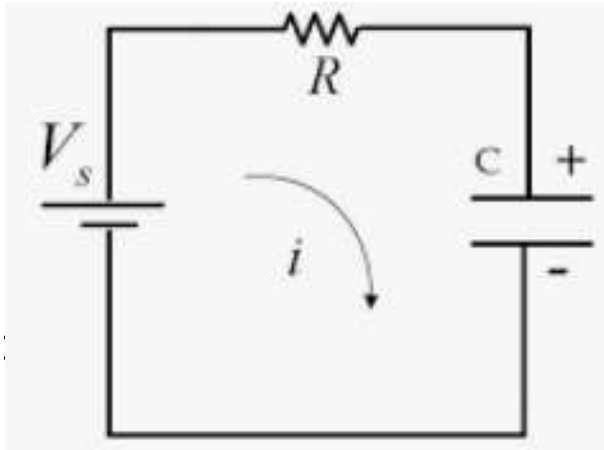
Model from physical laws: electrical circuit

- Using Kirchhoff's voltage law, and resistor and capacitor dynamics, the RC circuit equations are:

$$V_s = Ri + V_c$$

$$i = \dot{Q} = C\dot{V}_c$$

where V_s is a voltage source, Q and V_c, the charge and voltage of the capacitor and i the current



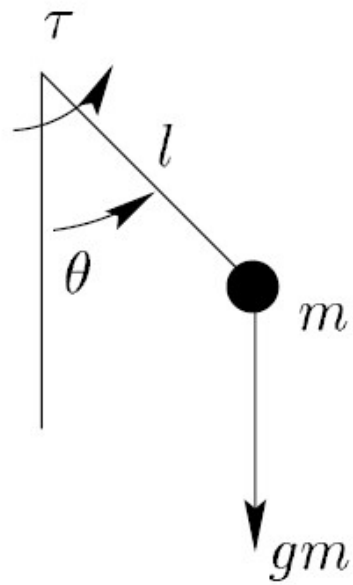
- Simplifying the equations by removing the current:

$$\dot{V}_c = \frac{1}{RC}(V_s - V_c)$$

- For constant source voltage, the solution is

$$V_c(t) = V_s(1 - e^{\frac{-t}{RC}})$$

Model from physical laws: pendulum



- Point-mass pendulum subject to gravity and torque

$$ml^2\ddot{\theta} = \tau(t) - mgl\sin(\theta)$$

$$\ddot{\theta} = -\frac{g}{l}\sin(\theta) + \frac{1}{ml^2}\tau$$

- Expressed as a first order system with state angle and rotation speed

$$\begin{aligned} \dot{\theta} &= \omega \\ \dot{\omega} &= -\frac{g}{l}\sin(\theta) + \frac{1}{ml^2}\tau \end{aligned}$$

Model simulation

- Solving system of differential equations requires numerical solvers: ODE solvers
- Standard representation $\dot{X} = F(t, X)$ used in the development and usage of ODE solvers
- User supplies the ODE solver the initial state $X(0)$, a function realizing F, and the time (or times) the solution is sought
- ODE solvers available in C, C++, FORTRAN, Python, OML, MATLAB, Octave, Scilab,..., based on libraries SUNDIALS, ODEPACK, LSODA,...
- ODE solvers may be used directly with little programming in the case of continuous-time models involving only differential equations

Numerical solution of ODEs

$$\dot{x} = f(t, x), \;\; x(0) = x_0$$

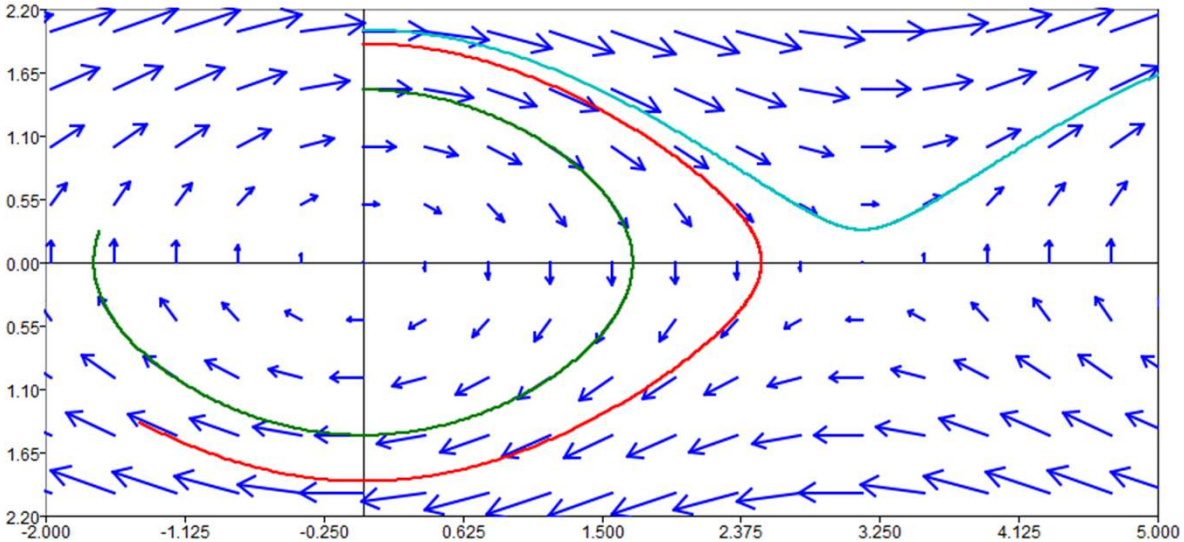
- Solving for *x(t)* over an interval *[0,T]*: need the values over the interval, not just *x(T)*
- *f(t,x)* provides the direction of *x* as *t* increases. Following this direction, the trajectory of *x(t)*, i.e., the solution of ODE, is obtained
- Very simple cases: analytical solution can be obtained. But in most realistic cases, the ODE must be solved numerically
- In 2D case, this trajectory can be visualized by drawing the vector field of the system. It shows the corresponding direction for various state values

Numerical solution of ODEs: example

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix} = \begin{pmatrix} x_2 \\ -\sin(x_1) \end{pmatrix}, \quad x(0) = \begin{pmatrix} a \\ b \end{pmatrix}$$

Vector field and trajectories for 3 initial conditions shown below

(a,b) = (0,1.5) , *(0,1.9)* and *(0,2.02)*



Numerical solution of ODEs: Euler method

$$\dot{x} = f(t, x), \ t \in [0, T]$$

Discretize interval $[0, T]$, $t_i = i.h$.

Taylor series expansion $x(t+h) = x(t) + h.\dot{x}(t) + \mathcal{O}(h^2)$, approximate the ODE by

$$x(t_{i+1}) = x(t_i) + h.f(t_i, x(t_i))$$

This method is explicit (new state is explicitly expressed as a function of previous state). Implicit version, backward Euler method:

$$x(t_{i+1}) = x(t_i) + h.f(t_{i+1}, x(t_{i+1}))$$

Better stability properties but implementation more complicated: an implicit equation must be solved at each time step. Newton method requires $\frac{\partial f}{\partial x}$.

Numerical solution of ODEs: higher order methods

Euler is a first order method. It can be seen as replacing the derivative by a first order discrete approximation

$$\dot{x}(t_i) \approx \frac{x(t_{i+1}) - x(t_i)}{h} = f(t_i, x(t_i))$$

One may consider using a higer order approximation of $\dot{x}$, for example

$$\dot{x}(t_i) \approx \frac{-x(t_{i+2}) + 4x(t_{i+1}) - 3x(t_i)}{2h} = f(t_i, x(t_i))$$

yielding the solver

$$x(t_{i+2}) = 4x(t_{i+1}) - 3x(t_i) - 2h.f(t_i, x(t_i))$$

But this recursion is unstable. Consider trivial case $\dot{x} = 0$, $x(0) = \epsilon > 0$. x grows unbounded.

Numerical solution of ODEs: Runge-Kutta methods

Evaluate f outside "meshpoints" for better estimation of derivative.

Explicit RK-2:

$$\begin{aligned} \tilde{x}(t_{i+1/2}) &= x(t_i) + \frac{h}{2} f(t_i, x(t_i)) \\ x(t_{i+1}) &= x(t_i) + h f(t_{i+1/2}, \tilde{x}(t_{i+1/2})) \end{aligned}$$

Explicit RK-4:

$$x(t_{i+1}) = x(t_i) + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

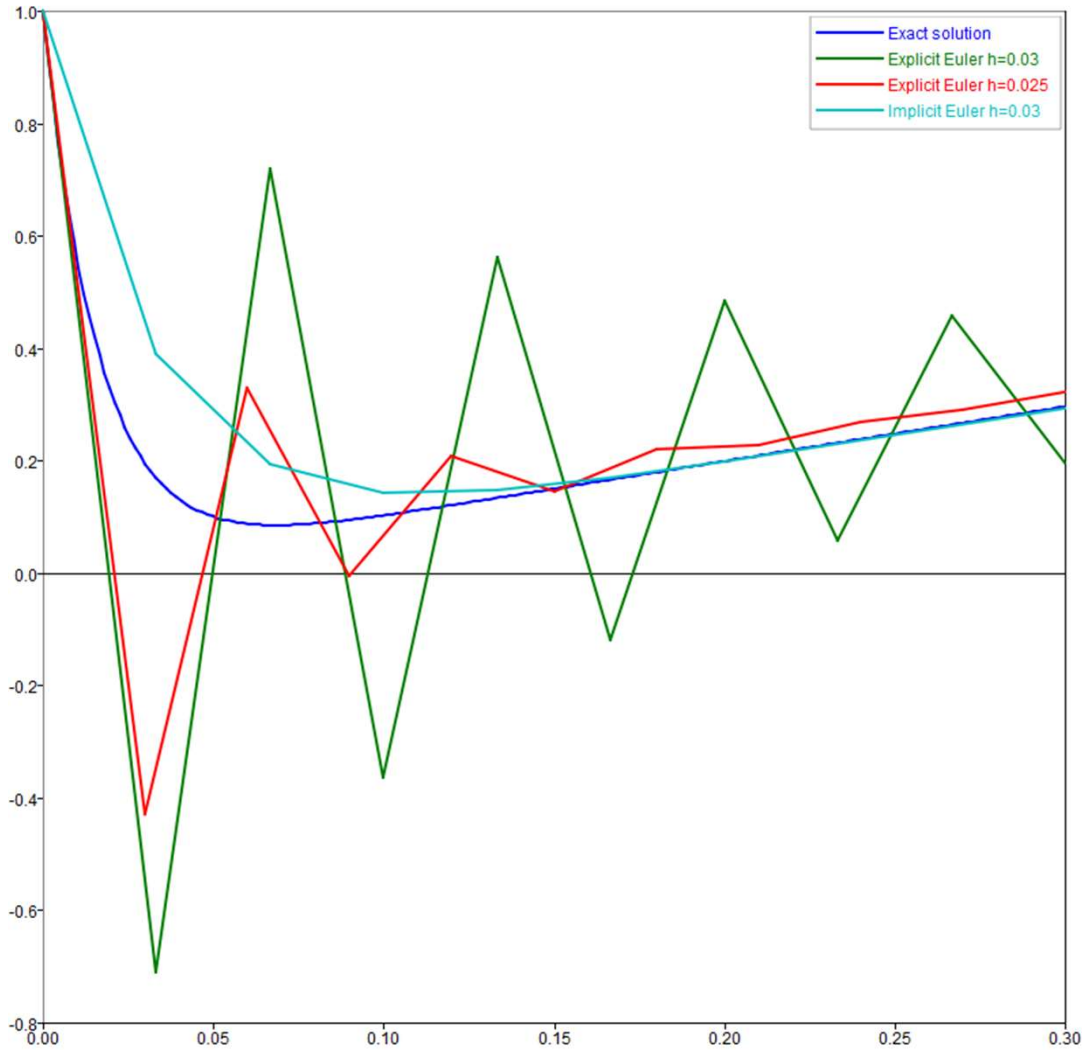
where

$$\begin{aligned} k_1 &= f(t_i, x(t_i)) \\ k_2 &= f(t_{i+1/2}, x(t_i) + \frac{h}{2} k_1) \\ k_3 &= f(t_{i+1/2}, x(t_i) + \frac{h}{2} k_2) \\ k_4 &= f(t_{i+1}, x(t_i) + h k_3) \end{aligned}$$

Numerical solution of ODEs: Explicit vs Implicit methods

- Implicit methods in general have better stability and accuracy. The step size h can be taken to be larger
- But implicit systems require solving an (in general) nonlinear system, so can be costly with risk of failing to find a solution
- Implicit systems particularly adapted for solving stiff systems. example

$$\dot{x} = -60(x - \sin(t)) + \cos(t)$$
$$\text{with } x(0) = 1.$$



Numerical solution of ODEs: variable step-size solvers

- Step size h need not be constant: it can be adjusted during simulation for better performance
- Systems may require small step sizes during some periods of time but accepts large step sizes at other times without sacrificing accuracy
- Error is estimated on-line, the step size is increased if the error bounds are satisfied. Steps are rejected and smaller steps taken if not.
- In most cases, variable step-size solvers should be used, not only because of performance but also error control. Fixed step-size solvers should be used when real-time performance is required (flight simulator,…)

Numerical solution of ODEs: Hot vs cold restart

- Hot/cold restart: numerical solvers estimate some parameters during simulation which are not systematically recomputed at every step for efficiency based on the assumption that the systems is smooth. These estimates constitute the state of the solver.
- When the simulation is interrupted, for example to read new input values, the last state of the solver can be used when the simulation restarts (hot restarts), or it can be reset (cold restart).
- The Hot vs Cold choice depends on whether the interruption has caused “non-smoothness” in the equations of the system. Hot restart should be considered in the smooth case because cold restart is expensive. But incorrect Hot restart can be costlier or even result in failure

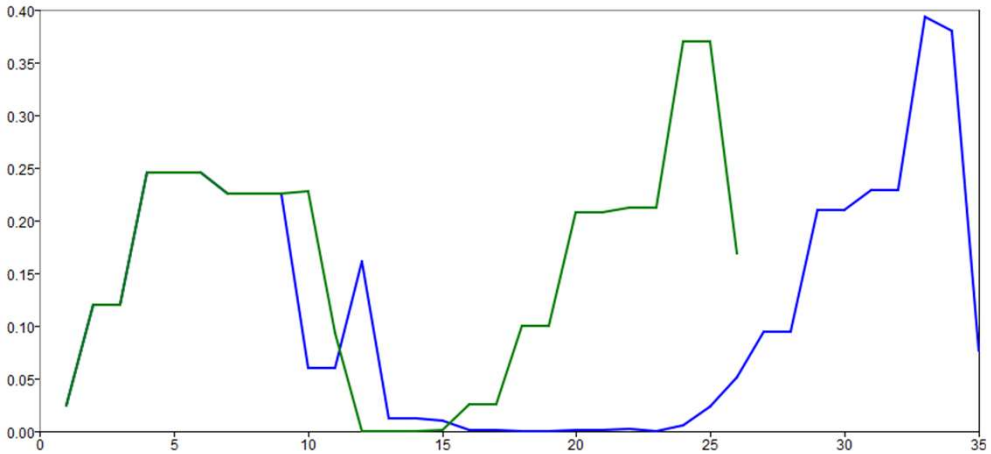
Numerical solution of ODEs: Hot vs cold restart example

Consider the non-smooth ODE

$$\dot{x} = \begin{cases} t\sin(x), & \text{if } t < 2 \\ t\sin(x) + 1, & \text{otherwise} \end{cases}$$

Comparing the time steps taken by solver in two cases:

- Integrate over interval [0,4] (blue curve)
- Integrate over interval [0,2], then do a cold restart and integrate over [2,4]



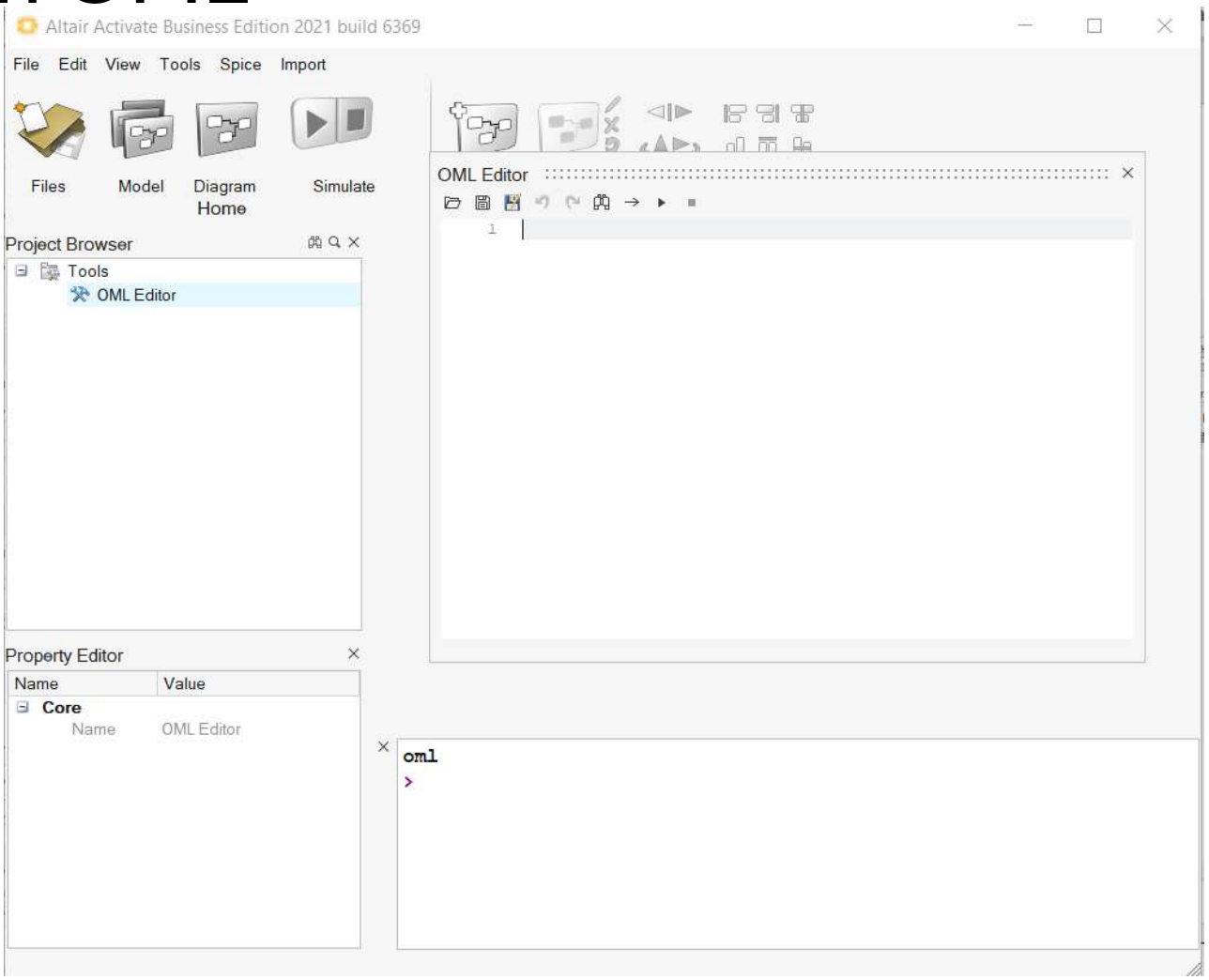
- Non-smoothness here occurs at t=2
- Non-smoothness may depend on state x, in which case zero-crossing test must be used

Numerical solution of ODEs: other considerations

- Numerical solvers are complex programs. It is important to understand their properties, but rarely a new solver needs to be developed for a specific problem
- High performance robust solvers are available in most scientific software such as Matlab, Scilab, OML, Octave, Julia, Python,…, or indirectly in graphical environments such as Simulink, Scicos, Twin Activate, Modelica…
- Some simulation environments manage automatically, to some extent, certain solver parameters, however users often need to make choices to improve performance or precision.

Model simulation in OML

- Open Twin Activate
- Open OML Editor from the Tools menu
- OML function ode45 used to solve ODE systems
- Information can be obtained on functions by help command in OML command window



Model simulation: Falling object

$$\dot{y} = v$$
$$\dot{v} = -g$$

Already first order form: state is $X = (y,v)$

Need to create a function F in the standard form to use ode45 to solve $\dot{X} = F(t, X)$

ode45: Solve a system of differential equations: uses variable step-size and implicit and explicit methods

Simple syntax:

- **[t,y] = ode45(@func,tin,y0)**

func: OML function: system of equations to solve.

tin: vector of times at which to report the solution.

y0: The vector of initial conditions.

OML Editor

```
function xd = F(t,x)
    g = 9.81;
    y=x(1);v=x(2);
    yd = v;
    vd = -g;
    xd = [yd,vd];
end

y0 = 10;
v0 = 0;
ic=[y0,v0];
tspan = linspace(0,1,100);
[tsol,xsol] = ode45(@F,tspan,ic);

plot(tsol,xsol)
```

Model simulation: Falling object with bounce

Same problem but when the ball reaches the ground, it bounces. Let *te* be the time of impact. At time *te*, the ball velocity points upward and loses 20% of its speed:

$$v(te) := -0.8v(te-) \text{ when } y(te)=0$$

ode45: zero-crossing event detection option. Must provide, in addition to the definition of the ODE, a function to define the zero-crossing surface

<u>syntax:</u>

- **[t,y,te,ye,ie] = ode45(@func,tin,y0,options)**

If a zero-crossing occurs, the exact time of crossing and the corresponding state are returned:

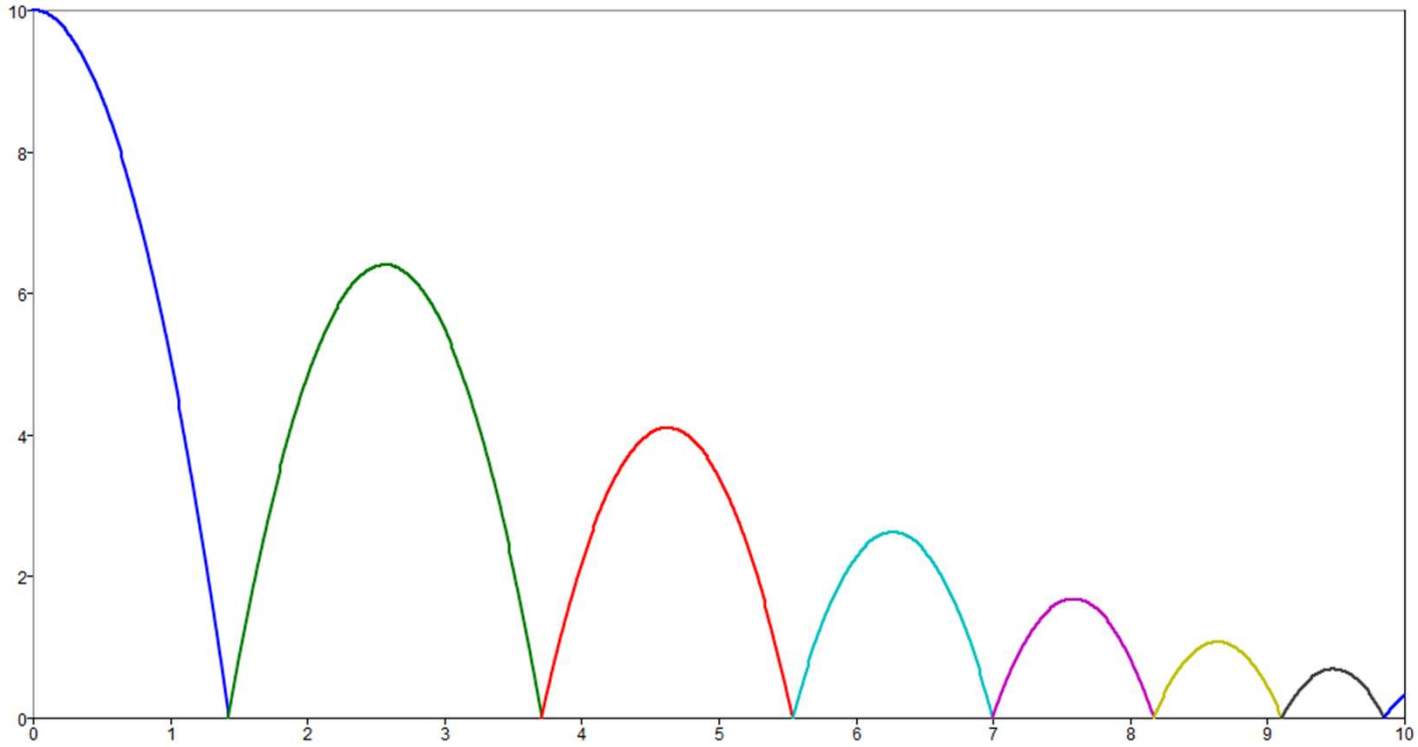
te: time of zero-crossing.

ye: state of ODE at time of zero-crossing.

```
function xd = F(t,x)
  g = -9.81; y = x(1); v = x(2);
  yd = v; vd = g; xd = [yd,vd];
end
function [value, isterminal, direction] = bounce(t, x)
    value = x(1); %stop when y crosses zero
    isterminal = 1; % Stop the integration
    direction = -1; % Detect crossings going negative
end
% Set the options
options = odeset('MaxStep',.01,'Events', @bounce);
y0 = 10; v0 = 0;
ic = [y0,v0]; T=10;
tspan = [0,T];
[tsol,xsol, te, xe, ie] = ode45(@F,tspan,ic,options);
plot(tsol,xsol(:,1))

while ~isempty(te)
  ic=xe'
  ic(2)=-.8*ic(2)
  tspan=[te,T];
  [tsol,xsol, te, xe, ie] = ode45(@F,tspan,ic,options);
  hold on
  plot(tsol,xsol(:,1))
end
```

Model simulation: Falling object with bounce

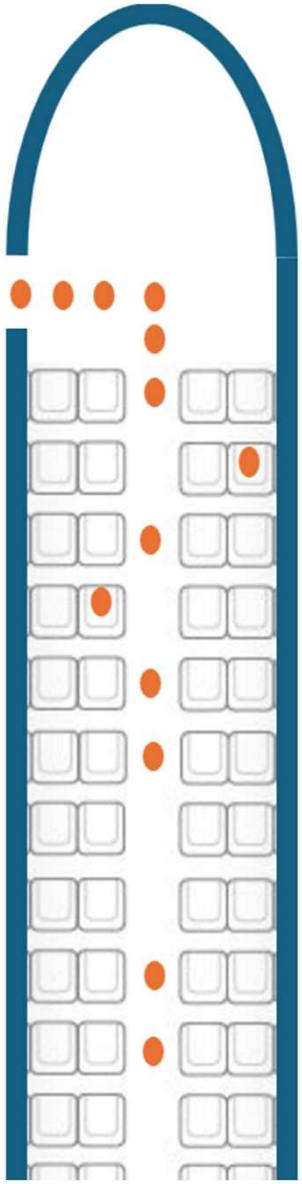


The position of the ball as a function of time

Model simulation: plane boarding

Simulating and evaluating boarding strategies on an airplane

- Example: boarding 120 passengers on a plane with one aisle, 30 rows of seats with 4 seats per row
- Passengers enter the plane aisle, one at a time, at row 1
- Only one passenger can be in the aisle next to each row
- Takes one unit of time for passenger to move forward one row (when no passenger blocking)
- When arrived at their row, a passenger will spend *d* units of time to prepare to sit. Only when seated, aisle becomes free



Compare different boarding strategies

Does it make boarding faster by organizing the boarding by calling passengers in stages based on their seat numbers?

Compute average boarding time in the following cases:

- Passengers enter plane in random order
- Passengers split in two groups: those with seats on rows 16 to 30, and those with seats on rows 1 to 15. The former group boards first
- Passengers board in the reverse order of their row numbers: passengers on the last row board first, then passengers on the row 29, etc.)

Choosing a state

- Each passenger is either: not entered the aisle, is in the aisle but has not reached their row, is in the aisle at their row waiting to be seated, or already seated
- No need to keep track of passengers already seated, for others must keep row number, current position, time spent at own row
- ***pass***: a matrix with two columns. First column contains the assigned rows. So, ***pass(5,1)*** is the assigned row of passenger 5. The second column keeps track of how long the passenger has been waiting next to their row.
- ***pos*** is a column vector representing the current position of the passengers. So, if ***pass(5,1)=pos(5)***, passenger 5 has reached their assigned row.
- When a passenger reaches their assigned row, they must wait ***d*** time units before seating. At that time, the passenger is deleted from pass and pos. The simulation ends when pass and pos are empty.

State update function for one time step

```
function [pos,pass]=next_step(pos,pass,d)
  % used by the simulator at each step
  i=0;
  while i<size(pass,1)
     i=i+1;
     if pass(i,1)==pos(i)  % passenger next to their seat
        if pass(i,2)==d % can now sit (already waited d)
             pass(i,:)=[];pos(i)=[];
             i=i-1;   % remove passenger and position
        else
             pass(i,2)=pass(i,2)+1; % not ready to sit
        end
     else
        if i==size(pos,1) || pos(i)+1<pos(i+1)
            pos(i)=pos(i)+1;   % passenger steps forward
        end
     end
   end
end
```

Every time step, passengers with space ahead of them can move forward a row. Since passengers are listed in the order they enter, to check if a passenger can step forward, only the position of the next passenger can be checked.

Simulation function

```
function t = boarding_time(pass,d,t)
   % originally all passengers are outside
   pos=zeros(size(pass)); % position 0: just outside plane
   pass=[pass,zeros(size(pass))];
   while ~isempty(pos)
        t=t+1;
        [pos,pass]=next_step(pos,pass,d);
    end
end
```

- Argument **pass** is a vector indicating the row numbers of the passengers entering the plane
- A second column of zeros is added to **pass** for tracking waiting times at the aisle
- Simulator repeatedly calls the update function until no passenger left

Boarding time for 3 Scenarios

Fully ordered case is deterministic (single run)

Time for other scenarios obtained by averaging over 50 runs

Result:

Boarding time, fully ordered case: 869
Average boarding time, fully random case: averaged over 50 simulations = 408.28
Average boarding time, 2 groups case: averaged over 50 simulations = 426.94

```
function [pass,Iper] = permrand(pass)
  % random permutation of input vector pass
  [Rows, Cols] = size(pass);
  r = rand(Rows, Cols);
  [xx,Iper] = sort(-r);
  pass=pass(Iper);
end
```

```
d=5;  % time to sit
pass=ones(4,1)*[1:30];pass=pass(:);

% Scenario 1: fully ordered
pass=-sort(-pass);  % sort in reverse order
t=0;
t = boarding_time(pass,d,t);
printf('Boarding time, fully ordered case: %g\n',t);

% Scenario 2: passengers boarding randomly
N=50;  % averaged over N simulations
t=0; %initial time
for k=1:N
  pass=ones(4,1)*[1:30];pass=pass(:);
  pass=permrand(pass);
  t = boarding_time(pass,d,t);
end
printf(['Average boarding time, fully random case: averaged\n',...
        'over %d simulations = %g\n'],N,t/N)

% Scenario 3: passengers boarding in two groups
t=0;
for k=1:N
  pass1=ones(4,1)*[1:15];pass1=pass1(:);pass1=permrand(pass1);
  pass2=ones(4,1)*[16:30];pass2=pass2(:);pass2=permrand(pass2);
  pass=[pass1;pass2];
  t = boarding_time(pass,d,t);
end
printf(['Average boarding time, 2 groups case: averaged\n',....
 'over %d simulations = %g\n'],N,t/N)
```

TD1: Introduction to Dynamical system modeling and script-based simulation

Exercise 1: Explicit and Implicit Euler: stability

Consider the following system

$$\dot{x} = -\lambda x + c, \;\; x(0) = K, \;\; \lambda > 0$$

- Find the analytic solution *x(t)* of this ODE
- Find the equation corresponding to the Explicit (forward) Euler solver for this system. Take the step size to be h. Under what conditions is this Explicit Euler solution stable and its solutions converges to the solution of the ODE?
- Consider now an Implicit Euler solver for the same system. Under what conditions is the Implicit Euler solution stable and converges to the solution of the ODE?

Exercise 2: Explicit and Implicit Euler: stiff systems

Consider the following ODE

$$\dot{x} = -60(x - \sin(t)) + \cos(t), \;\; x(0) = 1$$

over the interval $[0,0.3]$.

- Develop an OML code to construct a solution of this problem using the Explicit (forward) Euler solver
- Same problem for the Implicit (backward) Euler method.
- Show that the solution to this problem can be expressed as

$$x(t) = \exp(-60t) + \sin(t)$$

and compare it with the solutions obtained by Euler methods

Exercise 3: SIR model and simulation in OML, parametric study of SIR to assess lockdown

Consider the SIR model over [0,T]

$$\dot{s} = -as(t)i(t),$$
$$\dot{i} = as(t)i(t) - bi(t),$$
$$\dot{r} = bi(t),$$

- *s(t):* percentage of *susceptible* individuals
- *i(t)*: percentage of *infected* and contagious individuals
- *r(t)*: percentage of *recovered* individuals

- *a, b*: model parameters

- Typical values: *b=0.3, a=0.5 … 10, T=60*
- Initially *r=0*, *i* very small and *s=1-i*

Objective: develop an OML script for simulating this system for various values of parameter *a* and compare the results

- Use for state, $X=(s,i,r)$, for the system expressed as $\dot{X} = F(t, X)$
- Develop an OML function *sir(t,x,a)* returning the state derivative
- Function *sir(t,x,a)* cannot be used directly by ode45, which expects two argument t and x. Use closure as shown below to define the OML function F used by ode45 (note that the current value of a is "stored" in the definition F when this instruction is executed):

```
F = @(t,x) sir(t,x,a);
```

- Complete the script by looping over a range of a parameter values and plot the simulation result. To superpose plotted curves using the plot function use the OML command

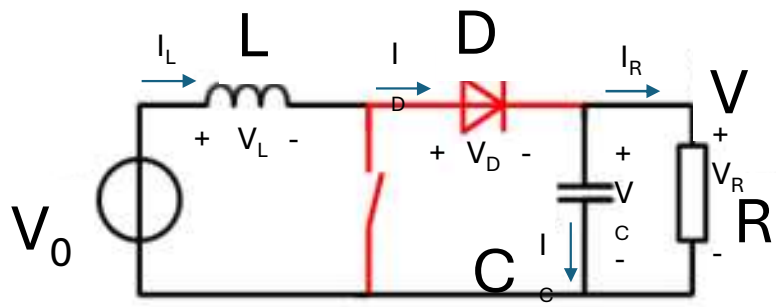
```
hold on
```

Exercise 4: SIR model and simulation in OML, study timing strategies of lockdown

Consider same SIR model as in previous exercise to study the effect of lockdown timing.

- Add parameters to sir function to include the ability to use:
  - two different values of the parameter a: $a=an$ corresponding to the normal situation, and $a=al$ during lockdown,
  - two time-instants T1 and T2. The lockdown is supposed to be applied from T1 to T2.
- Run simulations for various values of *T1* and *T2*. Comment on the consequences of the lockdown on the peak of infection (use the following values of a: $an = 1;\ al = 0.6;$

Exercise 5: Boost converter



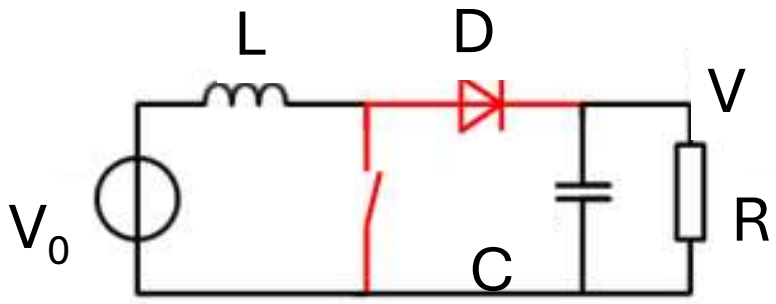
Consider this circuit where $V_0 > 0$ is a constant voltage source

The switch is periodically opened and closed: the frequency and duty cycle are given

Currents through and voltages across the inductor, diode, capacitor and resistor are denoted: I_L , I_D, I_C , I_R, V_L , V_D, V_C and V_R. Then

$$V_L = L.\dot{I}_L,\ I_C = C.\dot{V}_C,\ V_R = R.I_R,\ -V_D, I_D \geq 0 \text{ and } V_D.I_D = 0$$

Objective: Compute the output voltage $V(t) = V_R$, by simulation



- By taking as states I_L and V_C, find first order models of the system when the switch is closed and when it is opened (assume $V_R \geq 0$, to be verified a posteriori).
- Assume the switch is closed for a period T_1 then opened for a period T_2, then closed again for a period T_1, etc. Develop and OML script to simulate the model and plot the output voltage V. Note that the state values are continuous and do not jump when the switch is opened and closed
- Run simulations for various values of model parameters C, L, R, T_1, T_2 (typical values $L=1e\text{-}5$, $C=0.001$, $R=1K$, $T_1 = T_2 = 0.01$) and discuss the utility of such a circuit (assume $V_0 = 5$ Volts)